Step 5: Create a simple system instruction

If the Chat interface exposes a system prompt field, enter something uncomplicated:

"You are a concise technical assistant. Separate facts from assumptions. If you do not know something, say so rather than inventing an answer."

A system instruction does not change the weights; it simply guides behaviour for that conversation.

Step 6: Run the baseline prompt

Paste the battery prompt and send it.

Watch three things. How long does the model take before the first visible output? How quickly does the response stream once generation begins? How closely does it obey the requested 150-word structure and bullet points?

Also watch memory and processor usage in LM Studio or your operating system.

Save the response. We will compare it with the other runtimes.

Step 7: Change one setting, not ten

Try changing temperature if LM Studio exposes it prominently. Lower values generally make generation more conservative and predictable; higher values can make it more varied. Run the identical prompt again and see whether you can detect a meaningful difference.

Then put the setting back.

Local runtimes expose many controls, but random knob-turning makes troubleshooting harder. Change one variable at a time.

Step 8: Prove it is local

Once the model, runtime and anything else LM Studio needs have been downloaded, disconnect Wi-Fi or unplug Ethernet. Start a fresh chat and send the baseline prompt again.

LM Studio documents that downloaded models, document chat and its local server can operate without internet connectivity; model search, new downloads, runtime downloads and update checks still require connectivity.

If the response appears while your machine is offline, the abstract idea of "local AI" has just become very concrete.

Step 9: Unload when finished

When you stop using the model, unload it from memory rather than assuming closing a chat has released every resource. LM Studio's model controls provide unloading, and its CLI supports `lms unload`.

The file remains on your SSD. Unloading simply gives the RAM or VRAM back to the rest of your computer.

Route 2: Ollama, the command line without the pain

Ollama runs on macOS, Windows and Linux and turns model management into a small set of memorable commands. The terminal initially looks less friendly than LM Studio, but the workflow quickly becomes fast and repeatable.

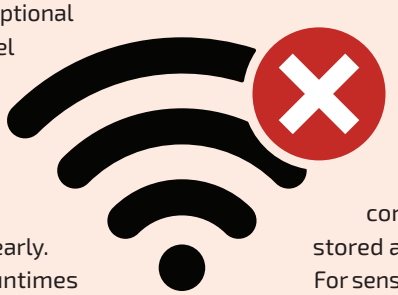
Step 1: Install Ollama

Install Ollama from its official download page for your operating system. Open Terminal, PowerShell or your preferred shell and type:

```
ollama
```

OFFLINE ISN'T THE SAME AS PRIVATE

"Runs locally" is often shortened to "private" or "offline", but those words describe different things. A model can perform inference locally while the surrounding application still has optional network features for updates, model search or integrations. Conversely, an application can be disconnected from the internet without being installed on a genuinely air-gapped computer.



LM Studio makes the distinction clearly. Once its model files and required runtimes are present, chatting with downloaded models can work offline, and the company says prompts entered into those local chats stay on the device. Its Discover search, model downloads, runtime downloads and app-update checks still need internet access. That is normal; an offline inference engine cannot magically obtain a new 5GB model.

Ollama now also supports both local and explicitly labelled cloud models. Do not assume that every model name appearing in the same tool has identical data flow. If privacy is the reason you are running locally, confirm that the exact model selected is stored and executed on your machine.

For sensitive work, test deliberately: download everything you need, disconnect networking, start a new session and verify that inference still works. "Local" should be a property you can demonstrate, not a sticker you take on faith.